```
import numpy as np

NUM_USERS = 5
TOTAL_BANDWIDTH = 100
NUM_PARTICLES = 30
MAX_ITERATIONS = 100
W = 0.7
C1 = 1.5
C2 = 1.5
ALPHA = 0.7
BETA = 0.3

channel_gain = np.array([1.2, 0.9, 1.5, 1.0, 0.8])

def fitness(position):
    bandwidth = np.maximum(position, 0)

    if np.sum(bandwidth) > TOTAL_BANDWIDTH:
        bandwidth = (
            bandwidth / np.sum(bandwidth)
            * TOTAL_BANDWIDTH
        )

    throughput = np.sum(
        channel_gain * np.log2(1 + bandwidth)
    )

    utilization = (
        np.sum(bandwidth) / TOTAL_BANDWIDTH
    )

    max_throughput = np.sum(
        channel_gain
        * np.log2(1 + TOTAL_BANDWIDTH)
    )

    normalized_throughput = (
        throughput / max_throughput
    )

    score = (
        ALPHA * normalized_throughput
        + BETA * utilization
    )

    return score, throughput, utilization

positions = np.random.uniform(
    0, TOTAL_BANDWIDTH / NUM_USERS,
    (NUM_PARTICLES, NUM_USERS)
)

velocities = np.random.uniform(
    -2, 2, (NUM_PARTICLES, NUM_USERS)
)

pbest = positions.copy()
pbest_scores = np.array([
    fitness(p)[0] for p in positions
])

best_index = np.argmax(pbest_scores)
gbest = pbest[best_index].copy()
gbest_score = pbest_scores[best_index]

for iteration in range(MAX_ITERATIONS):

    for i in range(NUM_PARTICLES):

        r1 = np.random.rand(NUM_USERS)
        r2 = np.random.rand(NUM_USERS)
```

```
        velocities[i] = (
            W * velocities[i]
            + C1 * r1 * (pbest[i] - positions[i])
            + C2 * r2 * (gbest - positions[i])
        )

        positions[i] += velocities[i]

        positions[i] = np.maximum(positions[i], 0)

        total = np.sum(positions[i])

        if total > TOTAL_BANDWIDTH:
            positions[i] *= TOTAL_BANDWIDTH / total

        score, _, _ = fitness(positions[i])

        if score > pbest_scores[i]:
            pbest[i] = positions[i].copy()
            pbest_scores[i] = score

    best_index = np.argmax(pbest_scores)
    if pbest_scores[best_index] > gbest_score:
        gbest = pbest[best_index].copy()
        gbest_score = pbest_scores[best_index]

best_score, throughput, utilization = fitness(gbest)

print("5G Network Resource Allocation using PSO")
print("----------------------------------------")
for i, bandwidth in enumerate(gbest):
    print(
        f"User {i + 1}: {bandwidth:.2f} MHz"
    )
print("----------------------------------------")
print(f"Total Bandwidth: {TOTAL_BANDWIDTH} MHz")
print(f"Allocated Bandwidth: {np.sum(gbest):.2f} MHz")
print(f"Throughput Metric: {throughput:.4f}")
print(f"Resource Utilization: {utilization * 100:.2f}%")
print(f"Best Fitness Score: {best_score:.4f}")
```

```
5G Network Resource Allocation using PSO
----------------------------------------
User 1: 22.33 MHz
User 2: 16.50 MHz
User 3: 28.17 MHz
User 4: 18.44 MHz
User 5: 14.56 MHz
----------------------------------------
Total Bandwidth: 100 MHz
Allocated Bandwidth: 100.00 MHz
Throughput Metric: 23.9177
Resource Utilization: 100.00%
Best Fitness Score: 0.7657
```

# Particle Swarm optimization (PSO)

## Pseudo Code

BEGIN

1. Define the objective function f(x)
2. Initialize parameters:
   Number of particles (N)
   Maximum interations (MaxIter)
   Inertia weight (w)
   Cognitive coefficient (c1)
   Social coefficient (c2)
3. Initialize particles:
   Generate random position and Velocities.
4. Evaluate the fitness of each particle.
5. Set each particle's personal best (pbest) to its current position
6. Set global best (gbest) to the best personal best among all particles
7. For each iteration from 1 to maxiter Do
   FOR each particle DO
   -> update velocity:
      $v = w * v + c1 * r1 * (pbest - x) + c2 * r2 * (gbest - x)$
   -> Update position:
      $x = x + v$.
   -> Evalute fitness f(x)
   -> If current fitness is better than pbest
      Update pbest = x

   End FOR
End FOR.

8. Display the Best Solution (gbest) and display it's function value of (gbest)

End;

## 5ej Network Resource Allocation

Allocate Bandwidth and Communication resource among users maximize throughput and improve the resource utlization.

Objective: To allocate the bandwidth and Communication resource among users and to maximizing network throughput and improving resource utlization by using PSO

objective function:

Maximize $F = \alpha T + \beta U$

T = Total Throughput

U = Resource Utlization

$\alpha, \beta$ = weight factor of or throughput and resource ~~Allocation~~ utlization.

## Pseudo Code

Begin

1. Define the objective function!
2. Initilize parameter.
   - No. of users (N)
   - Total available bandwidth (B)
   - No: of particle.
   - Maximum iterations (MaxIter)
   - Inertia weight (w)
   - Cognitive coefficient (c1)
   - Social coefficient (c2).

2. Initialize random bandwidth allocations for each particle.

3. Ensure that total allocated bandwidth does not exceed available bandwidth B.

4. Initilize random velocities for particles.

5. Evaluate the fitness of each particle:
   Calculate Total Throughput (T)
   Calculate resource utlization (U)
   Fitness = $\alpha \times T + \beta \times U$.

6. Set personal best (pbest) for each particles

7. Set global best (gbest) to the best particle's allocation.

8. for each iteration from 1 to MaxIter.
   Do,
   for each particle
   Do,
   -> Update velocity:
   $V = W * V + C1 \times r1 * (pbest - X) + C2 \times r2 * (gbest - X)$
   -> Update bandwidth allocation
   $X = X + V$
   -> Calculate fitness
   -> update pbest
   -> update gbest
   End For
   End for

9. Display; Best bandwidth allocation;
   maximum throughput
   Resource utlization

End.

output:

5 eg Network Resource Allocation using PSO

User 1 : 22.33 MHZ
User 2 : 16.50 MHZ
User 3 : 28.17 MHZ
User 4 : 18.44 MHZ
User 5 : 14.56 MHZ

Total Bandwidth : 100 MHZ
Allocated Bandwidth : 100.00 MHZ
Throughput Metric : 23.9177
Resource Utilization : 100.00 %.
Best Fitness Score : 0.7657

16/10/26
@IP Seen.